Name: Solomon Getu

Student ID: CTC-6335-26

Department: Cyber Secuirty

Given Task: Write write up for

OWASP Top 10 2025: Application Design Flaws

Platform: TryHackMe

Room: OWASP Top 10 2025: Application Design Flaws

Date: Aug 10, 2026 G.C

Monitor: Gedion Desalegn

I have covered these categories in my write up

- AS02 — Security Misconfiguration
- AS03 — Software Supply Chain Failures
- AS04 — Cryptographic Failures
- AS06 — Insecure Design

https://tryhackme.com/room/owasptopten2025two

1. Introduction

The OWASP Top 10 2025 Application Design Flaws room focuses on vulnerabilities that originate from weaknesses in application architecture, configuration, dependency management, cryptographic implementation, and security design.

While vulnerabilities that are purely implementation-related, these issues can originate from decisions made during the design and deployment stages of an application.

2. AS02 — Security Misconfigurations

https://tryhackme.com/room/owasptopten2025two?taskNo=2&sharerId=6a74a6c241b7fc97575e350b

2.1. General Overview

- Security misconfiguration occurs when an application, server, or service is deployed with insecure settings.
  - ✓ ***Common examples include:***
  - ✓ Exposed administrative interfaces
  - ✓ Default credentials
  - ✓ Unnecessary services
  - ✓ Excessive permissions
  - ✓ Debugging functionality enabled in production
  - ✓ Verbose error messages
  - ✓ Unrestricted APIs

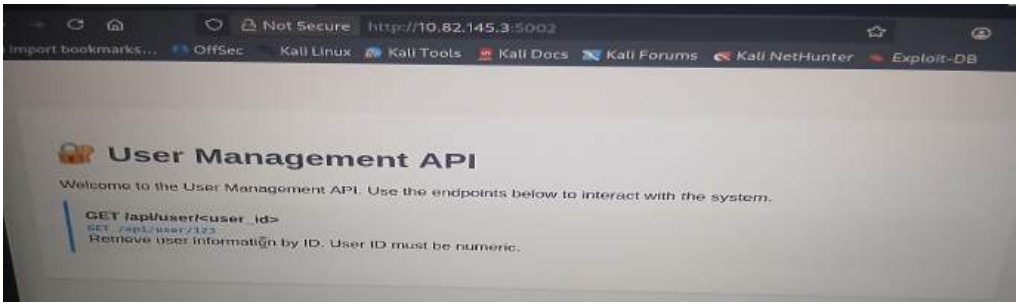
The hint that helps me to solve this challenge is

"Navigate to MACHINE_IP:5002. It appears that the developers left too many traces in their User Management APIs." which is clearly stated in the TryHackme lab room. When I solved this challenge the IP address was http://10.82.145.3:5002.

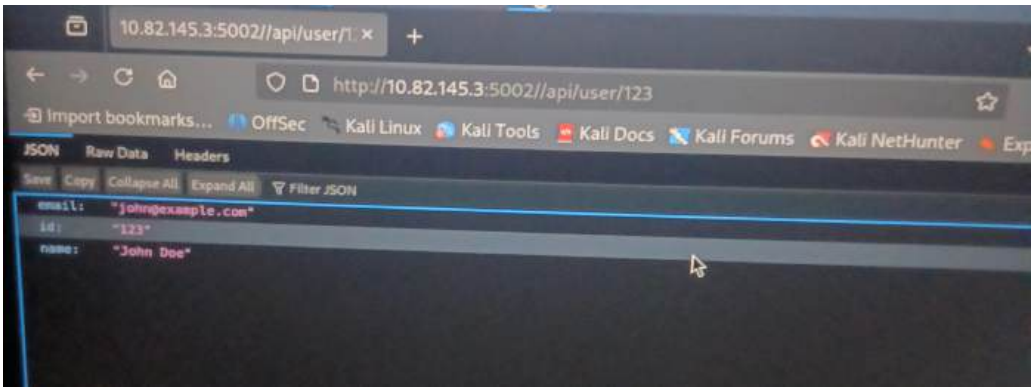
2.2. Walk-through Explanation

- First, I accessed the application running on:

http://10.82.145.3:5002



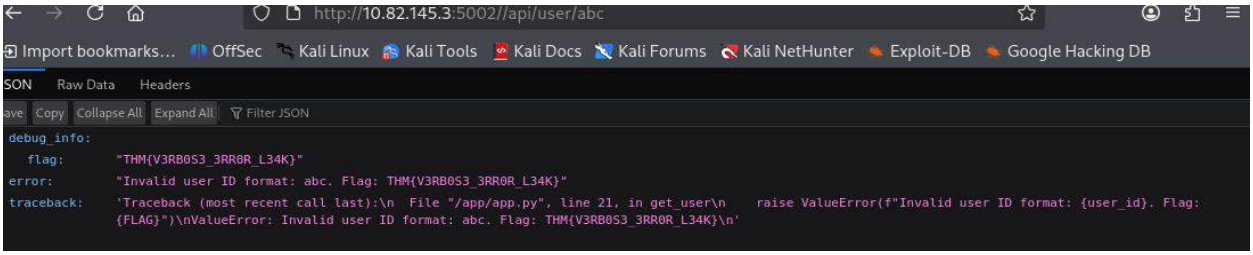
The application exposed a User Management API, and the API accepted a user_id parameter when I run this IP address on Firefox in their lab room. However, a normal request using a numeric identifier returned the expected response.



➢ Then I tested how the application handled unexpected input.

Instead of supplying a valid numeric user_id, I supplied a non-numeric value like

http://10.82.145.3:5002//api/users/abc



By the way, We can enter any kind of non numeric value it is just for test. As the result, the application returned an overly detailed error response. Therefore, this showed that the application's error-handling mechanism was ***improperly configured***.

2.3. Vulnerability

As a Vulnerability, the application exposed internal information through a verbose error message rather than returning a generic response such as, **“ Invalid user ID”**

As a result, the server disclosed information that should have remained internal.

2.4. Flag

✧ THM{V3RB0S3_3RR0R_L34K}



In this category I learned some Security Recommendations. For eaxmple;

Production applications should:

- Disable debug mode.
- Avoid exposing stack traces to users.
- Return generic error messages.
- Log detailed errors on the server instead.
- Review configuration before deployment.

3. AS03 — Software Supply Chain Failures

https://tryhackme.com/room/owasptopten2025two?taskNo=3&sharerId=6a74a6c241b7fc97575e350b

3.1. General Overview

❖ Software Supply Chain Failures occur when an application depends on vulnerable, outdated, compromised, or improperly verified components.

For example, Modern applications frequently depend on:

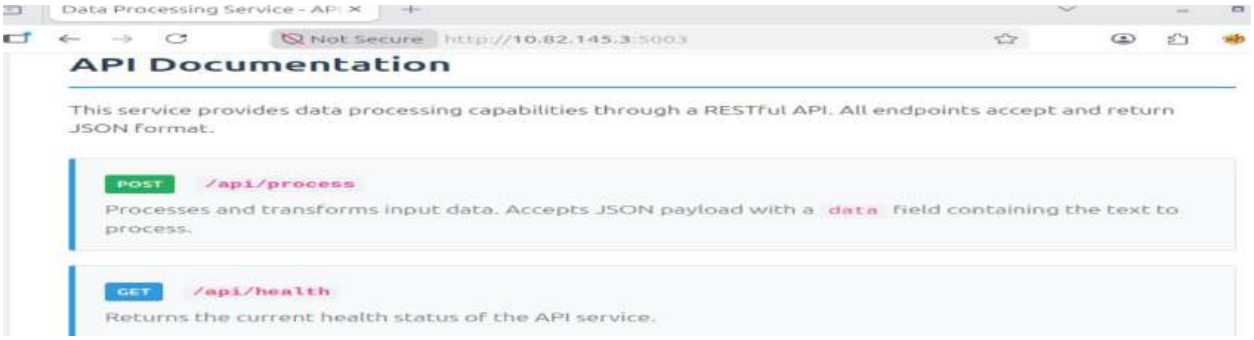
- ✓ Third-party libraries
- ✓ Open-source packages
- ✓ APIs (Application Programming Interfaces)

- ✓ Frameworks
- ✓ Container images
- ✓ AI/ML models

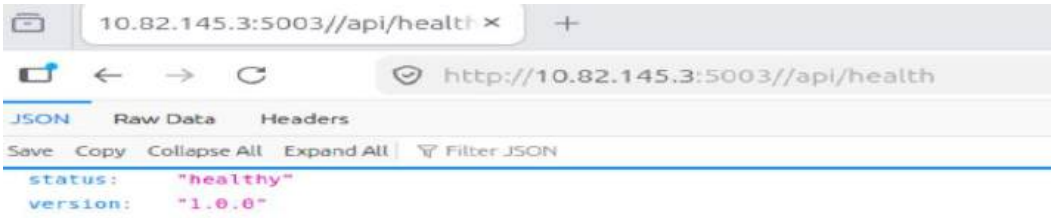
➢ Hint "Navigate to MACHINE_IP:5003. The code is outdated and import an old ***lib/vulnerable_utils.py*** component. Can you debug?" which is found when we opened the TryHackme lab room.

3.2. Source Code Analysis

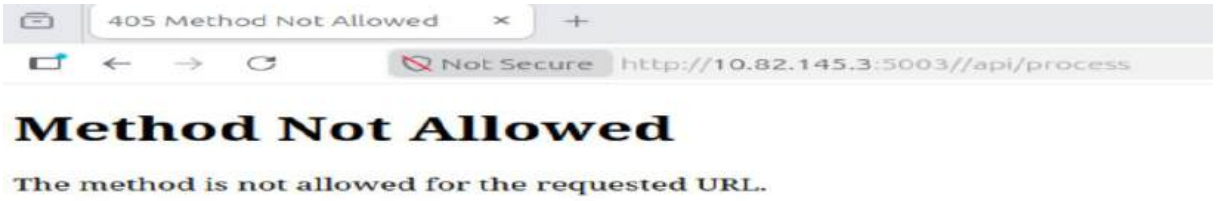
When I run the IP address for http://10.82.145.3:5003 on the browser, I got two API documentations which Post/api/process, and Get/api/health.



Get/api/health can give response,



but Post/api/process states method not allowed which means we have another possibility to open this source.



Therefore, The application exposed an API endpoint. As a result, I used burp suite and continue my

actions.





Finally, I edited the input, and run the request based on the given code clue. I wrote “POST /api/process”, and send request. In addition I used the given file as an important clue.

```
@app.route('/api/process', methods=['POST'])
def process():
    """Process user input using third-party library"""
    try:
        data = request.json.get('data', '')
        if not data:
            return jsonify({'error': 'Missing data parameter'}), 400

        # Check for debug mode
        if data == 'debug':
            return jsonify(debug_info())

        processed = process_data(data)
        formatted = format_output(processed)

        return jsonify({
            'result': formatted,
            'status': 'success'
        })
    except Exception as e:
        return jsonify({'error': str(e)}), 500
```

- The application expected JSON input containing a data parameter.
- The relevant logic contained a suspicious condition similar to:
- if “data” = “debug”:
- return jsonify(debug_info())

❖ **This was very interesting because a user-controlled value could trigger a debugging function. The imported functions came from *lib/vulnerable_utils.py***

3.3. Exploitation

- First I intercepted the request using Burp Suite Repeater.

The request was modified to:



- ✓ POST /api/process HTTP/1.1
- ✓ Host: MACHINE_IP:5003
- ✓ Content-Type: application/json
- ✓ {

"data"= "debug"

}

- Then the server processed the request and returned debugging information. Finally, the response displayed sensitive internal information, including the challenge flag.
- The important observation was that functionality originating from the vulnerable dependency was exposed through a production API. Independent walkthroughs confirm that sending data=debug to /api/process reveals the flag.

3.4. Vulnerability

- As a Vurnerability, the main issue is not simply that a debug function exists. The deeper problem is that the application trusted and exposed functionality from an outdated/unverified dependency and allowed a client-controlled value to invoke sensitive debugging functionality.

❖ Therefore, this demonstrated the risk associated with software supply chains.

3.5. Flag

✧ THM{SUPPLY_CH41N_VULN3R4B1L1TY}



What I learned regarding to Security Recommendations

Organizations/ Developers should:

- ✓ Maintain an inventory of dependencies.
- ✓ Keep dependencies updated.
- ✓ Scan dependencies for vulnerabilities.
- ✓ Remove unnecessary debugging functionality.
- ✓ Verify third-party components.
- ✓ Use Software Bills of Materials (SBOMs).
- ✓ Secure CI/CD pipelines.
- ✓ Monitor dependencies after deployment.

4. AS04 — Cryptographic Failures

https://tryhackme.com/room/owasptopten2025two?taskNo=4&sharerId=6a74a6c241b7fc97575e350b

4.1. General Overview

✓ **Cryptographic Failures** occur when sensitive information is not adequately protected because cryptography is missing, weak, incorrectly implemented, or improperly managed.

❖ Examples include:

- ✓ Hard coded encryption keys
- ✓ Weak algorithms
- ✓ Weak encryption modes

- ✓ Poor key management
- ✓ Unencrypted sensitive data
- ✓ Exposed secrets

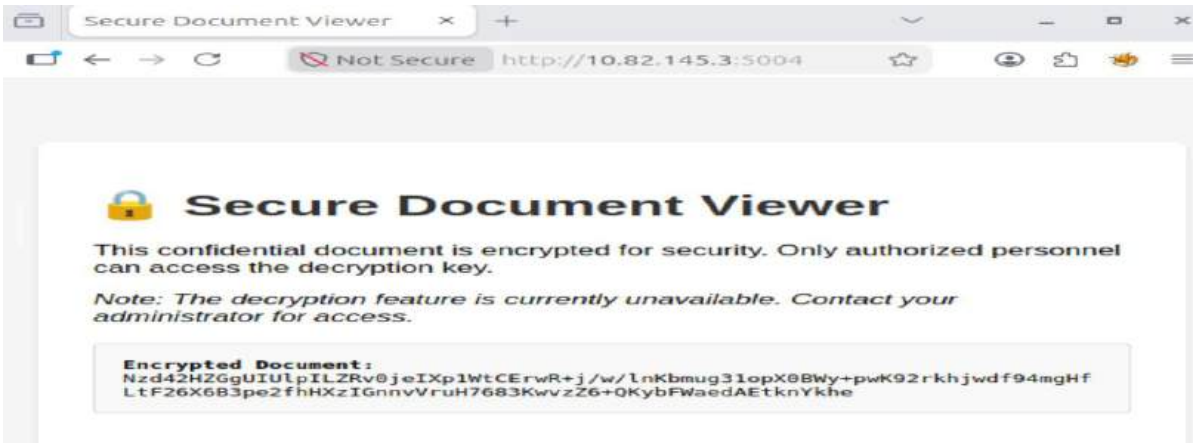
➢ Challenge

Navigate to MACHINE_IP:5004. Can you find the key to decrypt the file?, and what is the flag?

4.2. Inspecting the Application

✓ I opened: http://10.82.145.3:5004

✓ I inspected the page source and JavaScript files. A JavaScript file was referenced: /static/js/decrypt.js.



❖ This was significant because client-side JavaScript is accessible to anyone using the application.

4.3. Finding the Encryption Key

✓ Inspecting **decrypt.js** revealed the encryption implementation.

✓ The key was hard coded: "my-secret-key-16"

✓ The JavaScript implementation used AES in ECB mode.

```
from Crypto.Cipher import AES
from Crypto.Util.Padding import unpad
import base64

key = b"my-secret-key-16"

encrypted_base64 = "ENCRYPTED_DATA"

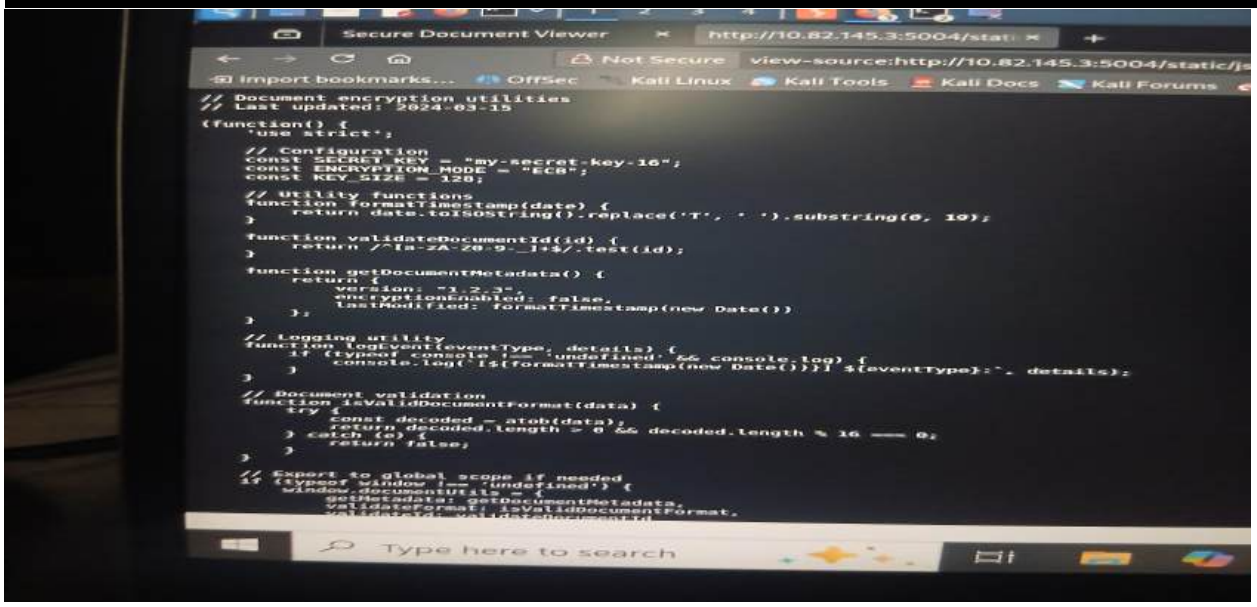
encrypted_data =
base64.b64decode(encrypted_base64)

cipher = AES.new(key, AES.MODE_ECB)

decrypted_padded =
cipher.decrypt(encrypted_data)

decrypted = unpad(decrypted_padded,
AES.block_size)

print(decrypted.decode())
```



- ❖ The important weakness was that the encryption key was stored directly in client-side code. Therefore, anyone capable of inspecting the application could retrieve it.
- ❖ A reference walk through confirms that the application used the hard coded key and AES-ECB implementation.

4.4. Decrypting the Data

- ➢ The encrypted data was Base64 encoded; therefore, the encryption process could be reproduced locally.

For example, using Python:



❖ The decrypted content contained the flag.

4.5. Vulnerability

➢ Several cryptographic weaknesses were present:

1. Hardcoded key

The encryption key was embedded in JavaScript:

my-secret-key-16

Therefore, the key was effectively public.

2. Client-side key storage

A secret required for decryption should not be distributed to every client.

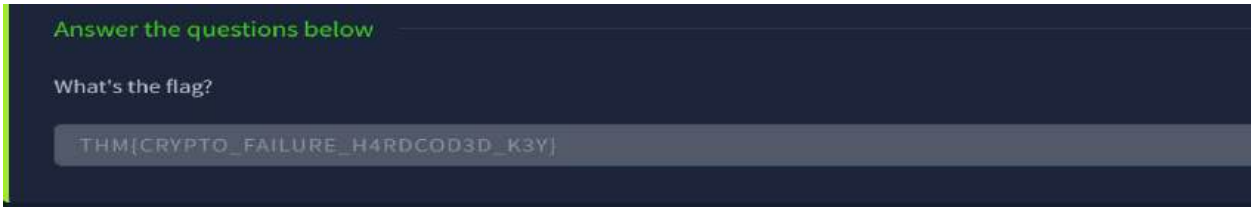
3. ECB mode

✓ AES-ECB is generally inappropriate for protecting structured sensitive data because identical plain text blocks produce identical ciphertext blocks.

Modern applications should use authenticated encryption modes such as AES-GCM.

4.6. Flag

✧ THM{CRYPTO_FAILURE_H4RDCOD3D_K3Y}



❖ ***What I learned as Security Recommendations***

✓ Never hardcode sensitive keys in client-side JavaScript.

✓ Store secrets using a secure secret-management system.

✓ Use modern authenticated encryption such as AES-GCM.

- ✓ Implement proper key rotation.
- ✓ Protect encryption keys separately from encrypted data.
- ✓ Perform regular cryptographic reviews.

5. AS06 — Insecure Design

https://tryhackme.com/room/owasptopten2025two?taskNo=5&sharerId=6a74a6c241b7fc97575e350b

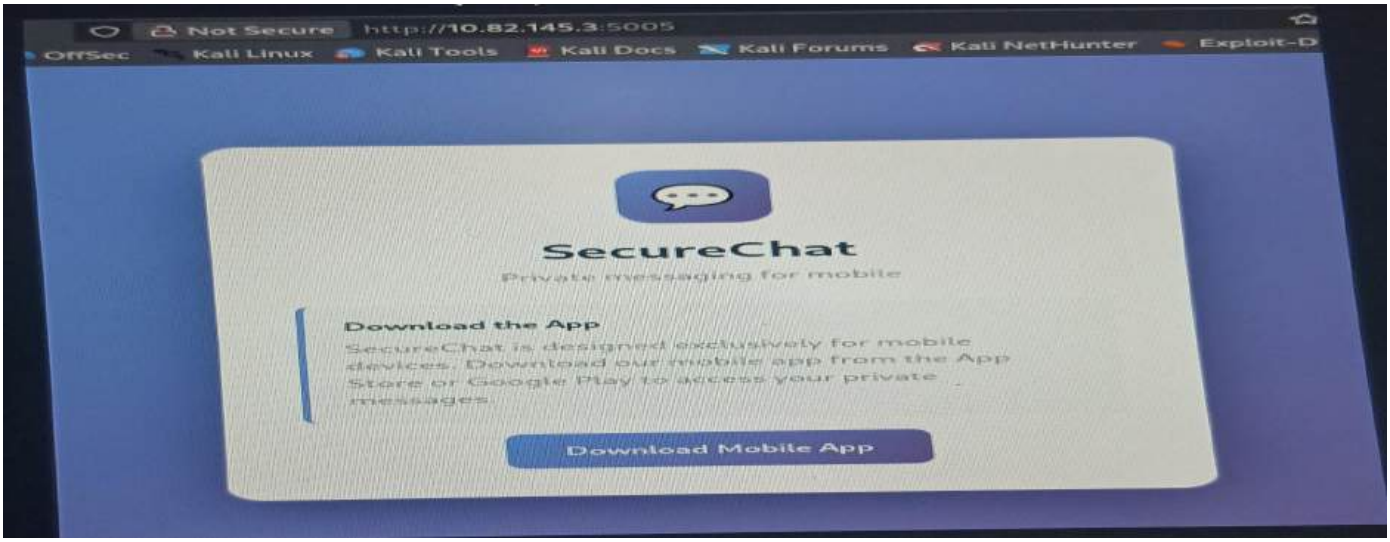
5.1. Overview

- ➢ Insecure Design occurs when security weaknesses are built into an application's architecture or business logic.
- ➢ The important distinction is that this is a design problem, rather than simply a coding mistake.
- ➢ OWASP describes insecure design as weaknesses resulting from missing or ineffective security controls and emphasizes threat modeling and secure design practices.
- ➢ The challenge asks:

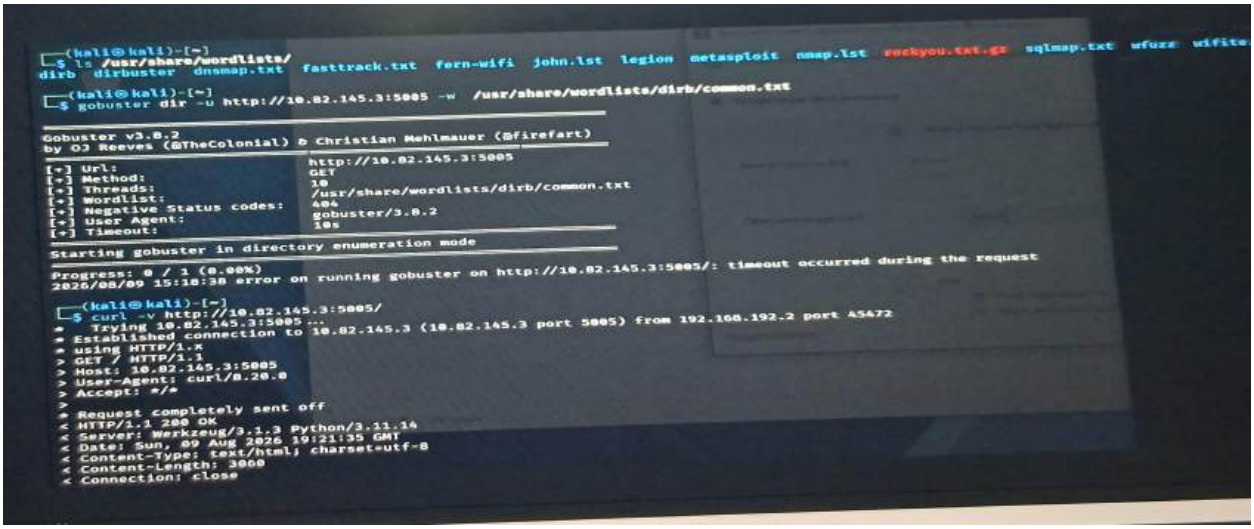
Navigate to MACHINE_IP:5005. Have they assumed that only mobile devices can access it?

5.2 Initial Enumeration

- ✓ First, I accessed: http://10.82.145.3:5005
- ✓ It displays the text states the application appeared to be designed as a private chat application.



✓ As a hint, the challenge suggested that the developers had assumed that only mobile devices would access the application.



❖ This is an important security assumption because information such as the **HTTP User-Agent** is controlled by the client.

Therefore:

■ ***User-Agent ≠ trustworthy authentication***

Which means changing the User-Agent does not prove that a request originated from a legitimate mobile application.

5.3. API Enumeration

✓ I investigated the application's API structure.

The API directory was:

/api

✓ Further enumeration revealed:

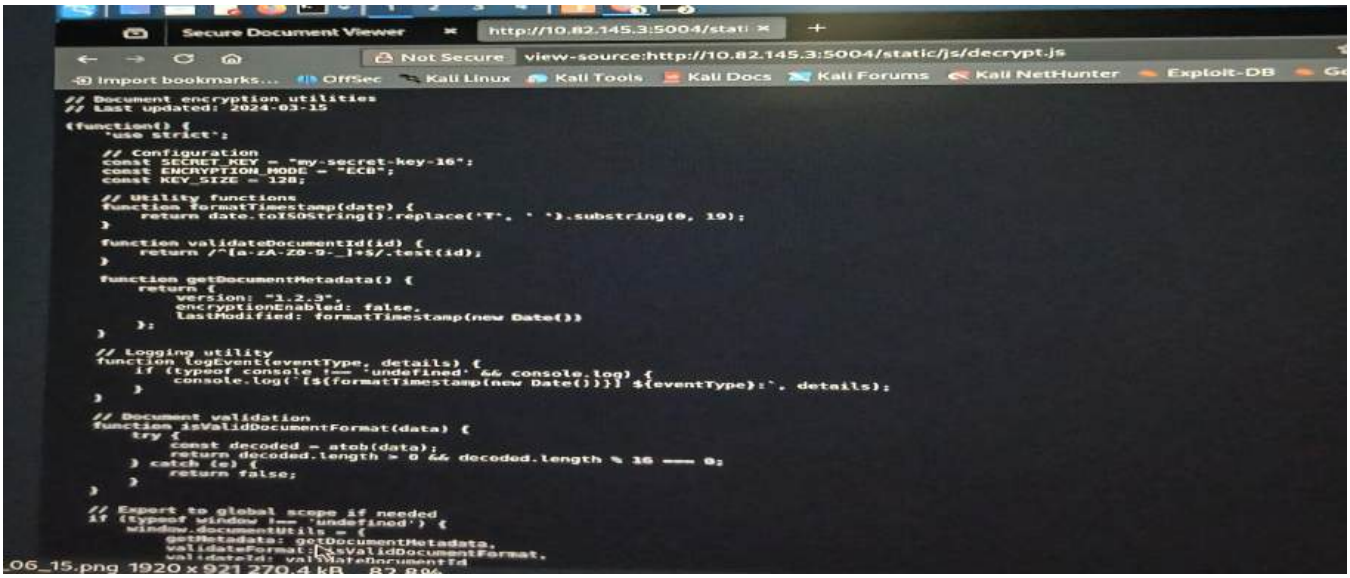
/api/users

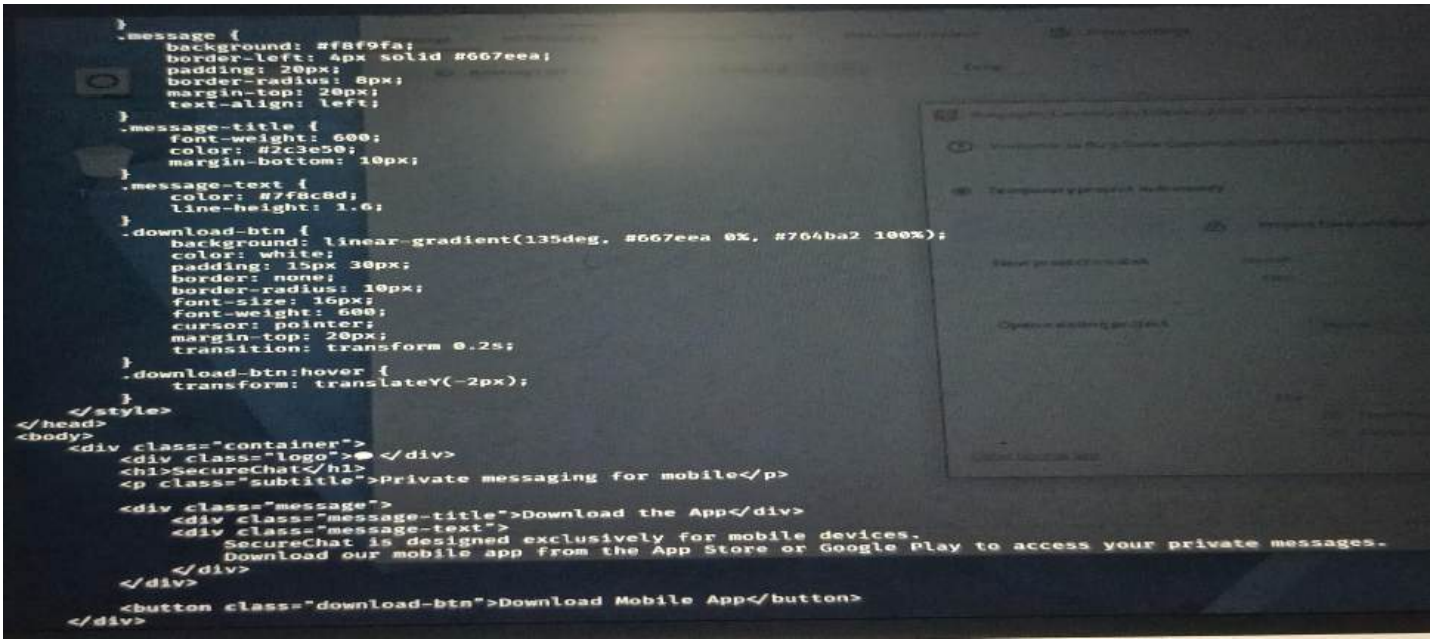
Additional endpoints exposed information associated with users.

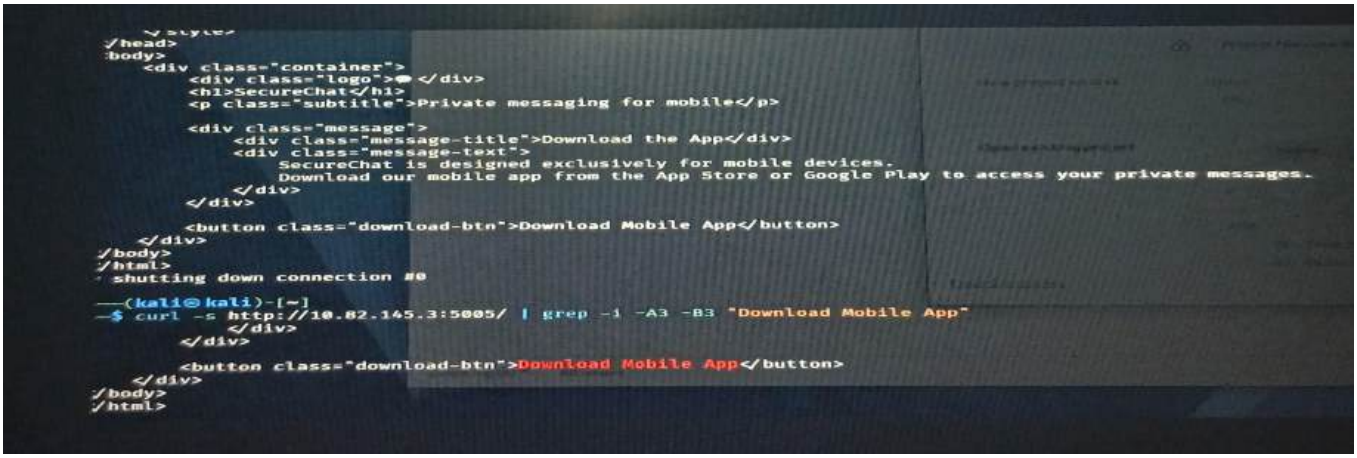
✓ For example:

- /api/users/admin
- /api/users/user1

- /api/users/user2



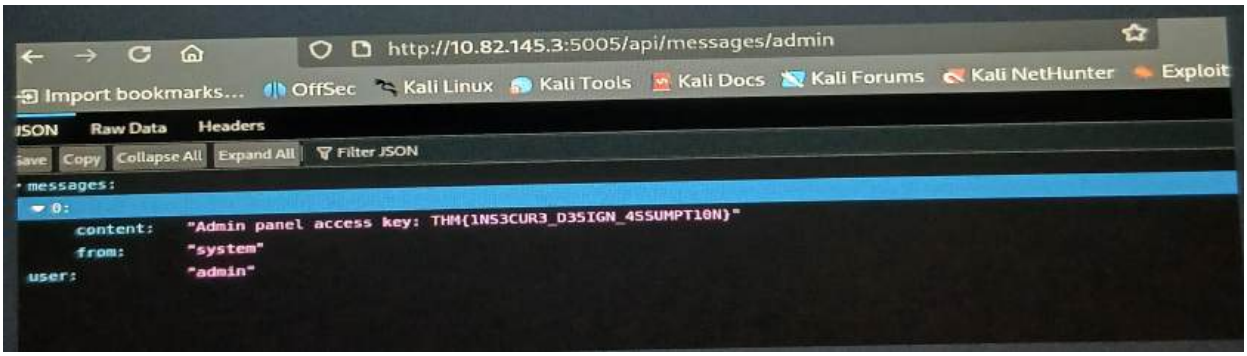




- ❖ This demonstrated that sensitive API functionality was accessible without proper authorization controls.

5.4. Discovering the Messages Endpoint

- ➢ The application was a private messaging system; consequently, I investigated whether a messages API existed.
  - ◆ The endpoint:
    - ● /api/messages was discovered.
  - ◆ Further enumeration revealed:
    - ● /api/messages/admin



- ➢ Accessing the endpoint returned the administrator's messages, which contained the flag.

5.5. Vulnerability

- ✓ The fundamental design flaw was the assumption that restricting access based on the expected client/device was sufficient. Therefore, the application effectively trusted client-

controlled information instead of implementing proper server-side authentication and authorization.

- ✓ The insecure assumptions included only mobile devices will access the API and users will not discover the API endpoints.
- ✓ These assumptions are not security controls.

5.6. Flag

- ✧ THM{1NS3CUR3_D35IGN_4SSUMPT10N}



What I learned regarding to Security Recommendations

- ❖ The application should:
  - ✓ Authenticate API requests.
  - ✓ Authorize every sensitive endpoint.
  - ✓ Never rely on User-Agent for security.
  - ✓ Apply server-side access controls.
  - ✓ Enforce least privilege.
  - ✓ Perform threat modeling before implementation.
  - ✓ Treat all client-controlled data as untrusted.

7. Conclusion

In general, this room demonstrates that serious application-security vulnerabilities can originate from decisions made before and around implementation, rather than from sophisticated exploitation techniques alone.